



# CS-250 : Lab 02

**Dynamic Memory Allocation with Git and Unit Testing**

**Muhammad Mujtaba**

**CMD ID: 540040**

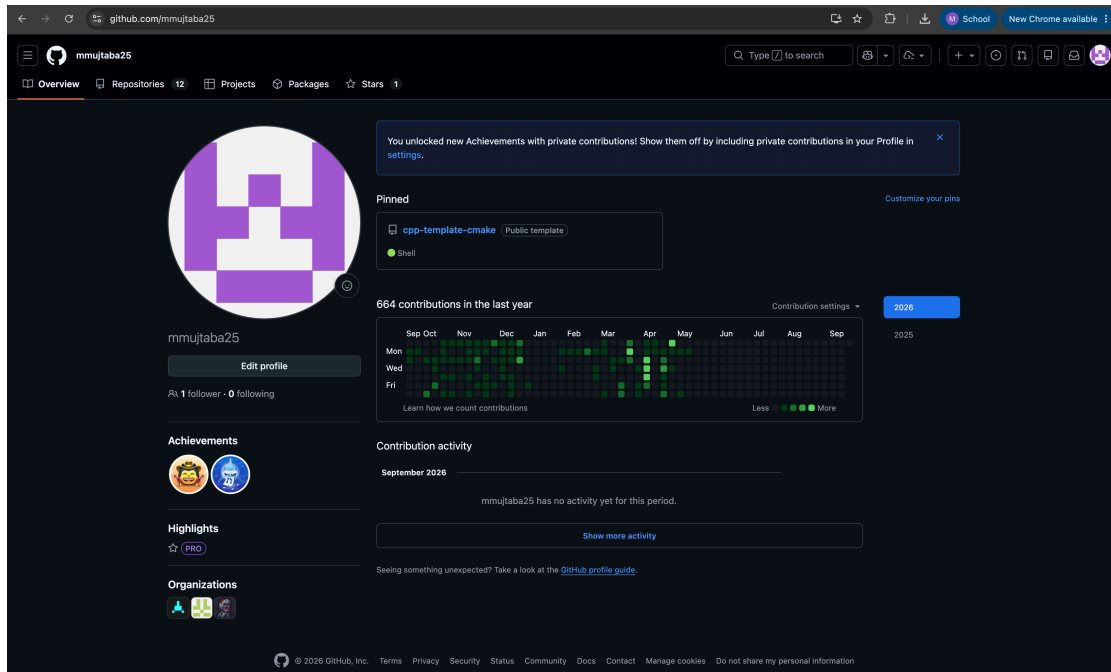
[mmujtaba.bese25seecs@seecs.edu.pk](mailto:mmujtaba.bese25seecs@seecs.edu.pk)

**Class: BESE 16B**

**Batch: 2k25**

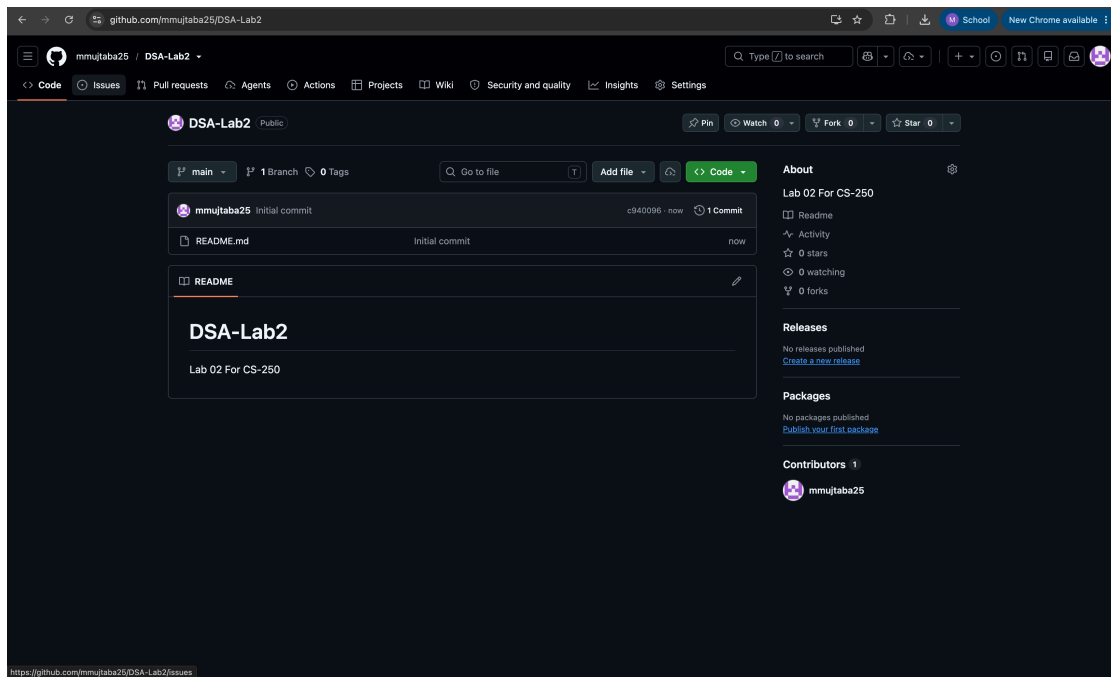
# Task 1:

## I. Creating a GitHub Account:



Link: <https://github.com/mmujtaba25>

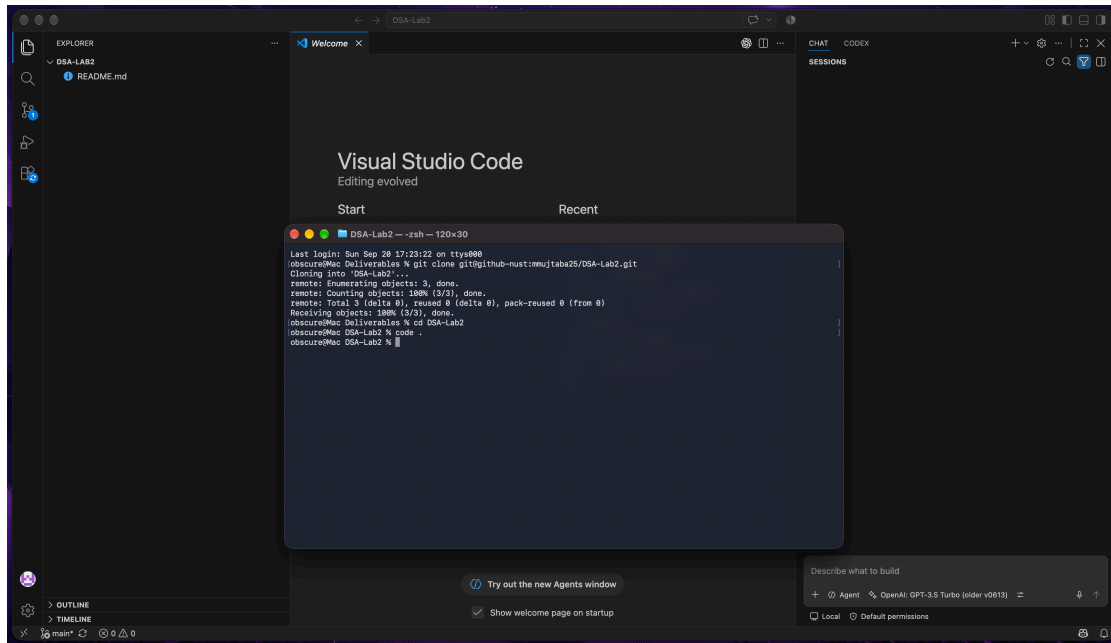
## II. Create a New Repository:



Link: <https://github.com/mmujtaba25/DSA-Lab2>

[mmujtaba.bese25seecs@seecs.edu.pk](mailto:mmujtaba.bese25seecs@seecs.edu.pk)

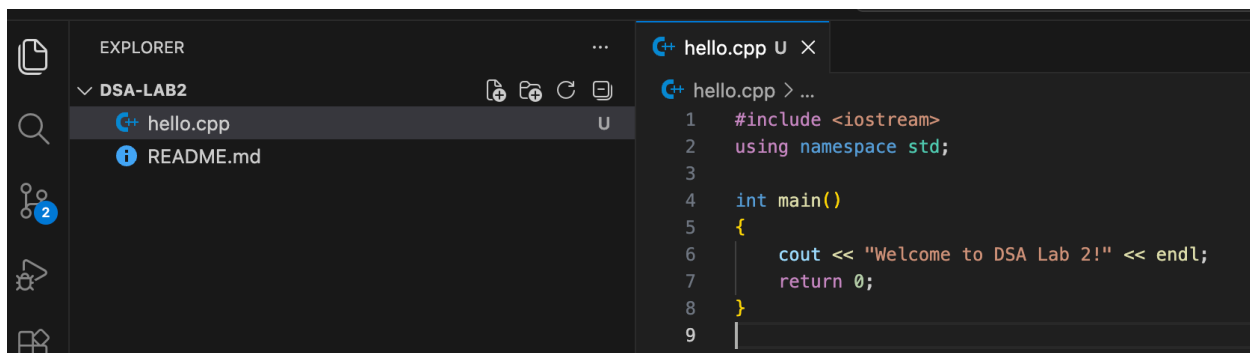
### III. Clone Repository to Local Machine:



### IV. Set Git Identity:

```
[obscure@Mac DSA-Lab2 % git config user.name
mmujtaba25
[obscure@Mac DSA-Lab2 % git config user.email
229266859+mmujtaba25@users.noreply.github.com
obscure@Mac DSA-Lab2 %
```

### V. Add a Starter Program:



## VI. Build and Run:

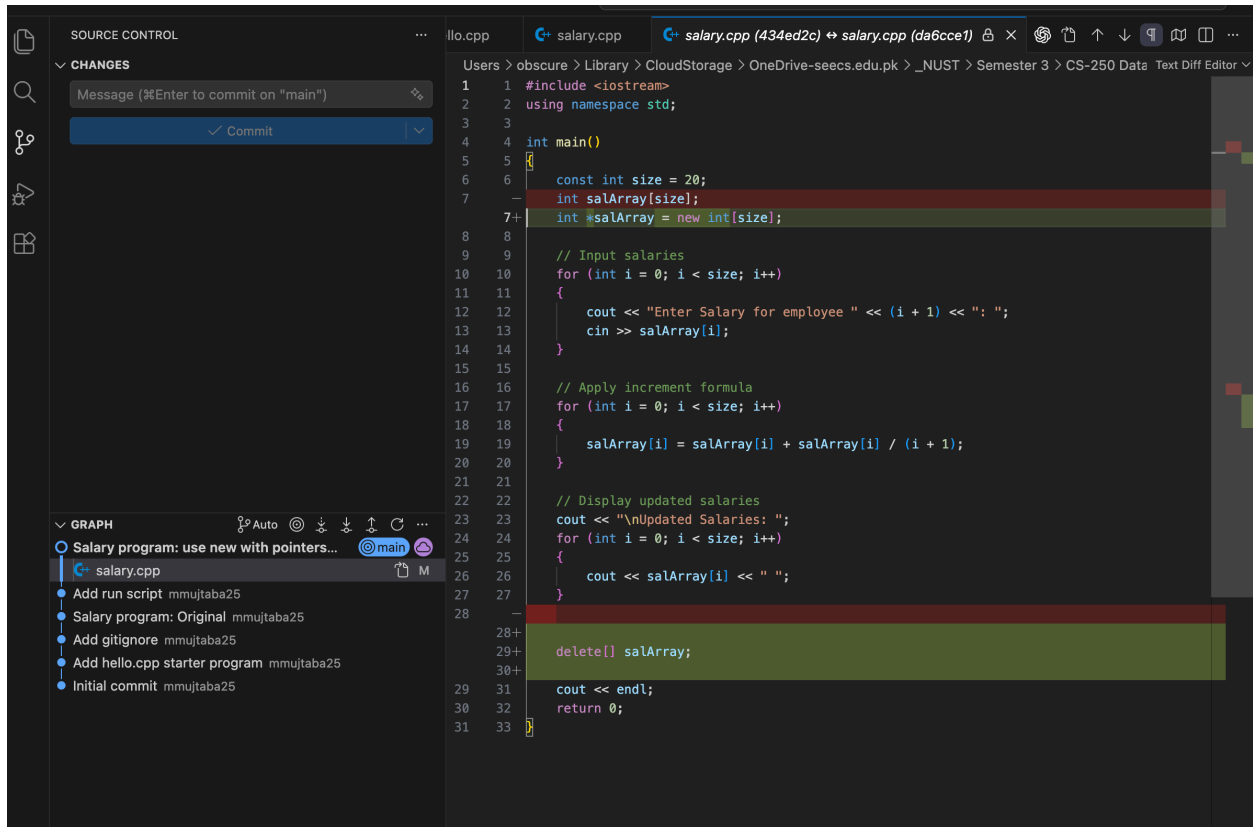
```
● obscure@Mac DSA-Lab2 % g++ ./hello.cpp -o hello
● obscure@Mac DSA-Lab2 % ./hello
Welcome to DSA Lab 2!
○ obscure@Mac DSA-Lab2 %
```

## VII. Stage, Commit, and Push:

```
● obscure@Mac DSA-Lab2 % git add hello.cpp
● obscure@Mac DSA-Lab2 % git commit -m "Add hello.cpp starter program"
[main 499caba] Add hello.cpp starter program
1 file changed, 8 insertions(+)
create mode 100644 hello.cpp
● obscure@Mac DSA-Lab2 % git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 408 bytes | 408.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github-nust:mmujtaba25/DSA-Lab2.git
c940096..499caba main -> main
○ obscure@Mac DSA-Lab2 %
```

## Task 2:

Updated Program Committed on Git:



```
1 1 #include <iostream>
2 2 using namespace std;
3 3
4 4 int main()
5 5 {
6 6     const int size = 20;
7 7     int salArray[size];
8 8     int *salArray = new int[size];
9 9
10 10 // Input salaries
11 11 for (int i = 0; i < size; i++)
12 12 {
13 13     cout << "Enter Salary for employee " << (i + 1) << ": ";
14 14     cin >> salArray[i];
15 15 }
16 16
17 17 // Apply increment formula
18 18 for (int i = 0; i < size; i++)
19 19 {
20 20     salArray[i] = salArray[i] + salArray[i] / (i + 1);
21 21 }
22 22
23 23 // Display updated salaries
24 24 cout << "\nUpdated Salaries: ";
25 25 for (int i = 0; i < size; i++)
26 26 {
27 27     cout << salArray[i] << " ";
28 28 }
29 29 delete[] salArray;
30 30
31 31 cout << endl;
32 32 return 0;
33 33 }
```

Link: <https://github.com/mmujtaba25/DSA-Lab2/commit/da6cce1b18a5746b91767e22b993613aefb6a4a0>

## Task 3:

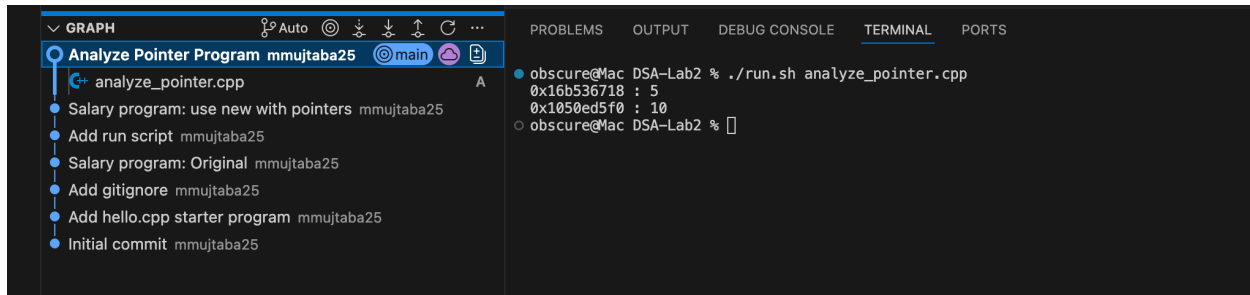
```
#include <iostream>

void analyze_pointer(int *ptr)
{
    std::cout << ptr << " : " << *ptr << "\n";
}

int main()
{
    int stackInt = 5;
    analyze_pointer(&stackInt);

    int *heapInt = new int;
    *heapInt = 10;
    analyze_pointer(heapInt);
    delete heapInt;

    return 0;
}
```



## Task 4:

```
#include <iostream>
#include <string>

constexpr int DEBUG = false;

namespace util
{
    void swap(char &a, char &b);
    int floor(double value);
    void debug_print(const char c);
    void debug_print(const char *str, int size);
}

void reverse_string(char *str, const int size);
void reverse_string_raw(char *str, const int size);

int main()
{
    constexpr int SIZE = DEBUG ? 11 : 51; // one extra for NULL byte
    char *str = new char[SIZE];

    std::cout << "Enter a string (max " << SIZE - 1 << " chars): ";
    std::cin.getline(str, SIZE);

    reverse_string(str, SIZE);

    std::cout << "Your string is reverse is: " << str << "\n\n";
    delete[] str;

    return 0;
}

namespace util
{
    void swap(char &a, char &b)
    {
        a = a ^ b;
        b = a ^ b;
        a = a ^ b;
    }

    int floor(double value) { return (int)value; }

    void debug_print(const char c)
    {
        if (!DEBUG)
            return;

        // print ":" for NULL byte
        if (c == '\\0')
            std::cout << ":";

        std::cout << c;
    }

    void debug_print(const char *str, int size)
    {
        if (!DEBUG)
            return;

        for (size_t i = 0; i < size; i++)
            debug_print(str[i]);

        std::cout << "\n";
    }
}
```

```

void reverse_string_raw(char *str, const int size)
{
    util::debug_print(str, size);

    if (DEBUG)
        std::cout << "\nMIRRORING: \n\n";

    // mirror string
    for (size_t i = 0; i < util::floor(size / 2); i++)
    {
        util::swap(str[i], str[size - i - 1]);
        util::debug_print(str, size);
    }
}

void reverse_string(char *str, const int size)
{
    reverse_string_raw(str, size);

    // get first null character from the right
    // first null from right is where cout terminates
    int null_at = 0;
    for (int i = size - 1; i >= 0; i--)
    {
        if (str[i] == '\0')
        {
            null_at = i + 1;
            break;
        }
    }

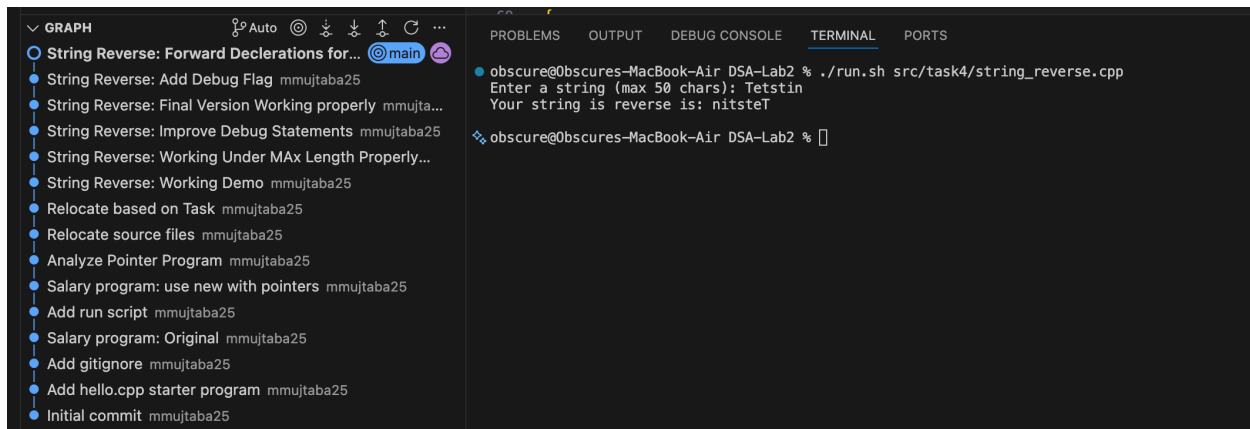
    if (DEBUG)
        std::cout << "\nOFFSETTING: \n\n";
    util::debug_print(str, size);

    // move string so str[0] is the last character before first character
    for (size_t i = 0; i < (size - null_at); i++)
    {
        util::swap(str[i], str[i + null_at]);
        util::debug_print(str, size);
    }

    // append NULL at string end (before str[0] in original string)
    str[size - null_at] = '\0';

    util::debug_print(str, size);
}

```



Link: [https://github.com/mmujtaba25/DSA-Lab2/blob/main/src/task4/string\\_reverse.cpp](https://github.com/mmujtaba25/DSA-Lab2/blob/main/src/task4/string_reverse.cpp)

## Task 5:

### I. Write the Tests First:

```
void testSortedArray()
{
    int arr[] = {1, 2, 3, 4, 5};
    assert(isSorted(arr, 5) == true);
}

void testUnsortedArray()
{
    int arr[] = {4, 5, 2, 3, 1};
    assert(isSorted(arr, 5) == false);
}

void testDuplicateValues()
{
    int arr[] = {1, 2, 3, 4, 4};
    assert(isSorted(arr, 5) == true);
}

void testSingleElement()
{
    int arr[] = {1};
    assert(isSorted(arr, 1) == true);
}

void testDescendingArray()
{
    int arr[] = {5, 4, 3, 2, 1};
    assert(isSorted(arr, 5) == false);
}

void testNegativeValues()
{
    int arr[] = {-5, -4, -3, -2, -1};
    assert(isSorted(arr, 5) == true);
}

void testAllSameValues()
{
    int arr[] = {1, 1, 1, 1, 1};
    assert(isSorted(arr, 5) == true);
}

void testEmptyArray()
{
    assert(isSorted({}, 0) == true);
}
```

## II. Implement the Function:

```
bool isSorted(const int *arr, const int size)
{
    if (size == 0)
        return true;

    for (size_t i = 0; i < size - 1; i++)
    {
        if (arr[i] > arr[i + 1])
        {
            return false;
        }
    }

    return true;
}
```

## III. Demonstrate the TDD Cycle:

Test for Empty Array Failing (the function fails before result can reach assert function)

```
⊗ obscure@Obscures-MacBook-Air DSA-Lab2 % ./run.sh src/task5/array_sort.cpp
./run.sh: line 6: 15106 Segmentation fault: 11 ./out/bin
● obscure@Obscures-MacBook-Air DSA-Lab2 % ./run.sh src/task5/array_sort.cpp
❖ obscure@Obscures-MacBook-Air DSA-Lab2 %
```

Before and After Commit “Array Sort: testEmptyArray Pass”

Link: <https://github.com/mmujtaba25/DSA-Lab2/commit/95302ebeb9bd7e7ac9ab50dcbb2ab18889c0892>

## IV. Git Workflow:

Tests were written in Commit Named “Array Sort: Write Tests”

Link: <https://github.com/mmujtaba25/DSA-Lab2/commit/5a664e6b5d298f0bf58984835659b9ef01791540>

